

Project Name: SentinelShield Advanced Linux Security Hardening & Threat Audit Framework

Promod Chandra Das -231002005
Chinmoy Debnath- 231902029

```
#!/bin/bash
# SentinelShield v1.1 Polished Final
# Audit + Policy PASS/FAIL + Safe SSH Hardening (Backup/Validate/Restart/Rollback) +
Report/Score/Log
# + CPU Scheduling (FCFS/SJF/RR) + renice demo
set -u

BACKUP_DIR="$HOME/sentinel_backup"
TMPDIR="/tmp"
POLICY_FILE="$BACKUP_DIR/sentinel_policy.ini"
LOGFILE="$BACKUP_DIR/sentinel.log"
SSH_CFG="/etc/ssh/sshd_config"

FIND_PRUNE=( -path /proc -o -path /sys -o -path /run -o -path /dev ) -prune -o'

# ----- UI tuning (reduces ugly look) -----
export NEWT_COLORS='
root=white,black
window=white,black
border=white,black
title=green,black
textbox=white,black
button=black,green
compactbutton=white,black
entry=white,black
label=cyan,black
actbutton=black,green
listbox=white,black
actsellistbox=black,cyan
checkbox=green,black
,

need() { command -v "$1" >/dev/null 2>&1 || { echo "Missing dependency: $1"; exit 1; }; }
have() { command -v "$1" >/dev/null 2>&1; }
is_root() { [ "$(id -u)" -eq 0 ]; }
ensure() { mkdir -p "$BACKUP_DIR"; touch "$LOGFILE" 2>/dev/null || true; }
log(){ ensure; echo "[$(date '+%F %T')] $" >> "$LOGFILE"; }
```

```

# ----- small UI helpers -----
msg() { whiptail --title "$1" --msgbox "$2" 12 86; }
bigmsg() { whiptail --title "$1" --msgbox "$2" 16 92; }
show() { whiptail --title "$1" --scrolltext --textbox "$2" 24 94; }
confirm() { whiptail --title "$1" --yesno "$2" 12 86; }

# ----- Policy -----
ensure_policy() {
    ensure
    [ -f "$POLICY_FILE" ] && return
    cat >"$POLICY_FILE"<<'EOF'
[SSH]
PermitRootLogin=no
PasswordAuthentication=no

[SCAN]
MaxWorldWritableFiles=50
MaxWorldWritableDirs=50
EOF
    log "Created policy $POLICY_FILE"
}

ini_get(){ # section key file
    awk -v S="$1" -v K="$2" '
        $0~"^[[:space:]]*\\[.\\.\\[[:space:]]*$"{in=(S==S);next}
        in && $0!~"^[[:space:]]*#" && $0!~"^[[:space:]]*$"{
            split($0,a,"="); k=a[1]; v=a[2]
            gsub(/^[[:space:]]+|[[:space:]]+$/,"",k); gsub(/^[[:space:]]+|[[:space:]]+$/,"",v)
            if(k==K){print v; exit}
        }
    ' "$3"
}

# ----- SSH detection -----
ssh_installed() { [ -f "$SSH_CFG" ] || have sshd || [ -x /usr/sbin/sshd ]; }

ssh_service_name() {
    systemctl list-unit-files 2>/dev/null | awk '{print $1}' | grep -qx "ssh.service" && { echo "ssh";
return; }
    systemctl list-unit-files 2>/dev/null | awk '{print $1}' | grep -qx "sshd.service" && { echo "sshd";
return; }
    systemctl is-active ssh 2>/dev/null >/dev/null && { echo "ssh"; return; }
    systemctl is-active sshd 2>/dev/null >/dev/null && { echo "sshd"; return; }
    echo ""
}

```

```

}

ssh_status() {
    local s; s=$(ssh_service_name)
    [ -n "$s" ] && systemctl is-active "$s" 2>/dev/null || echo "NOT FOUND"
}

ssh_restart() {
    local s; s=$(ssh_service_name)
    [ -n "$s" ] || return 1
    systemctl restart "$s" 2>/dev/null
}

sshd_bin() {
    have sshd && { command -v sshd; return; }
    [ -x /usr/sbin/sshd ] && { echo "/usr/sbin/sshd"; return; }
    echo ""
}

sshd_validate() {
    local b; b=$(sshd_bin)
    [ -n "$b" ] || return 1
    "$b" -t 2>/dev/null
}

sshd_get(){ # key -> last value
    [ -f "$SSH_CFG" ] || { echo ""; return; }
    awk -v K="$(echo "$1"|tr A-Z a-z)" '
        $0!~"^[[:space:]]*#" && tolower($1)==K{v=$2}
        END{print v}
    ' "$SSH_CFG"
}

sshd_set(){ # key value
    [ -f "$SSH_CFG" ] || return 1
    local k="$1" v="$2" tmp="$TMPDIR/sshd_config.sentinel.$$"
    awk -v K="$(echo "$k"|tr A-Z a-z)" '
        { if($0~"^[[:space:]]*#"){print; next}
          if(tolower($1)==K) next
          print
        }
    ' "$SSH_CFG" >"$tmp" || return 1
    echo "$k $v" >>"$tmp"
    cp "$tmp" "$SSH_CFG" && rm -f "$tmp"
}

```

```

}

backup_sshd(){
    ensure
    [ -f "$SSH_CFG" ] || return 1
    local ts; ts=$(date +%F_%H-%M-%S)
    cp "$SSH_CFG" "$BACKUP_DIR/ssh_config.$ts.bak" 2>/dev/null || return 1
    echo "$BACKUP_DIR/ssh_config.$ts.bak"
}

# ----- status line -----
status_line() {
    local r s
    r="USER"
    is_root && r="ROOT"
    s="SSH:N/A"
    ssh_installed && s="SSH:$(ssh_status)"
    echo "[r] $s Policy:$(basename "$POLICY_FILE") Log:$(basename "$LOGFILE")"
}

# ----- Intro + dependency check -----
intro() {
    ensure_policy; ensure
    local T="$TMPDIR/sentinel_intro.txt"
    {
        echo "SentinelShield v1.1"
        echo "Advanced Linux Security Hardening & Threat Audit Framework"
        echo
        echo "Purpose: Detect → Validate → Harden → Verify → Report"
        echo
        echo "Modules:"
        echo " • Security Audit (permissions, ports, suid/sgid, services)"
        echo " • Policy-based PASS/FAIL (INI policy)"
        echo " • Safe SSH Hardening (backup + validate + restart + rollback)"
        echo " • Security Report + Score + Logs"
        echo " • CPU Scheduling (FCFS/SJF/RR) + renice demo (OS Lab)"
        echo
        echo "-----"
        echo "$(status_line)"
    } >"$T"
    show "SentinelShield" "$T"
}

env_check() {

```

```

local T="$TMPDIR/sentinel_env.txt"
{
  echo "=== Environment Check ==="
  echo "Generated: $(date)"
  echo
  echo "Core:"
  echo " - whiptail: OK"
  echo
  echo "Audit tools:"
  have ss && echo " - ss: OK" || echo " - ss: MISSING (install iproute2)"
  have netstat && echo " - netstat: OK" || echo " - netstat: optional"
  echo
  echo "SSH hardening tools:"
  if ssh_installed; then
    echo " - OpenSSH Server: INSTALLED"
    echo " - sshd_config: $SSH_CFG"
    echo " - service: $(ssh_status)"
  else
    echo " - OpenSSH Server: NOT INSTALLED (Hardening/Rollback will be N/A)"
    echo "   Install:"
    echo "     sudo apt install openssh-server"
    echo "     sudo systemctl enable --now ssh"
  fi
  echo
  echo "Tip: Run with sudo for full coverage."
} >"$T"
show "Environment Check" "$T"
}

# ----- Audit -----
audit_basic_scan() {
  local T="$TMPDIR/sentinel_scan.txt"
  {
    echo "=== Basic Security Scan ==="
    echo "Generated: $(date)"
    echo "-----"
    echo "$(status_line)"
    echo
    echo "[+] World-writable files (top 10, REAL FS only):"
    local wf
    wf=$(eval find / $FIND_PRUNE -type f -perm -0002 -printf '%m %u:%g %p\n' 2>/dev/null |
head -n 10)
    [ -n "$wf" ] && echo "$wf" || echo "None found."
    echo
  }
}

```

```

    echo "[+] World-writable directories (top 10, REAL FS only):"
    local wd
    wd=$(eval find / $FIND_PRUNE -type d -perm -0002 -printf '%m %u:%g %p\n' 2>/dev/null |
head -n 10)
    [ -n "$wd" ] && echo "$wd" || echo "None found."
    echo
    echo "[+] Sticky-bit check (expected 1777 with 't'):"
    for d in /tmp /var/tmp /dev/shm /run/lock /run/screen; do
        [ -e "$d" ] && stat -c "%A %a %U:%G %n" "$d" 2>/dev/null || true
    done
} >"$T"
log "Audit: basic scan"
show "Audit - Basic Scan" "$T"
}

audit_ports() {
    local T="$TMPDIR/sentinel_ports.txt"
    {
        echo "=== Open / Listening Ports ==="
        echo "Generated: $(date)"
        echo "-----"
        echo "$(status_line)"
        echo
        if have ss; then
            ss -tulpen 2>/dev/null | head -n 70
        elif have netstat; then
            netstat -tulpen 2>/dev/null | head -n 70
        else
            echo "No ss/netstat found. Install iproute2 or net-tools."
        fi
    } >"$T"
    log "Audit: ports"
    show "Audit - Ports" "$T"
}

audit_suid() {
    local T="$TMPDIR/sentinel_suid.txt"
    local out
    out=$(eval find / $FIND_PRUNE -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | head -n
60)
    {
        echo "=== SUID/SGID Binaries (REAL FS only) ==="
        echo "Generated: $(date)"
        echo "-----"
    }

```

```

    echo "$(status_line)"
    echo
    [ -n "$out" ] && echo "$out" || echo "None found (or restricted permissions)."
} >"$T"
log "Audit: suid/sgid"
show "Audit - SUID/SGID" "$T"
}

audit_services() {
    local T="$TMPDIR/sentinel_services.txt"
    {
        echo "=== Services Overview ==="
        echo "Generated: $(date)"
        echo "-----"
        echo "$(status_line)"
        echo
        echo "[+] Running services (top 50):"
        systemctl list-units --type=service --state=running 2>/dev/null | head -n 50 || echo "systemctl
not available"
        echo
        echo "[+] Enabled services (top 50):"
        systemctl list-unit-files --type=service --state=enabled 2>/dev/null | head -n 50 || echo
"systemctl not available"
    } >"$T"
    log "Audit: services"
    show "Audit - Services" "$T"
}

audit_menu() {
    while true; do
        local c
        c=$(whiptail --title "Security Audit" --menu "$(status_line)\n\nSelect:" 20 86 10 \
            "1" "Basic Scan (permissions + sticky-bit)" \
            "2" "Open Ports Scan" \
            "3" "SUID/SGID Scan" \
            "4" "Services Overview" \
            "5" "Back" 3>&1 1>&2 2>&3) || return
        case "$c" in
            1) audit_basic_scan ;;
            2) audit_ports ;;
            3) audit_suid ;;
            4) audit_services ;;
            5) return ;;
        esac
    done
}

```

```

done
}

# ----- Hardening -----
policy_check() {
    ensure_policy
    local T="$TMPDIR/sentinel_policy.txt"
    {
        echo "=== Policy Validation (INI) ==="
        echo "Policy file: $POLICY_FILE"
        echo "Generated: $(date)"
        echo "-----"
        echo "$(status_line)"
        echo
        if ! ssh_installed; then
            echo "SSH NOT installed -> SSH checks are N/A"
            echo
            echo "Install to demo hardening:"
            echo "  sudo apt install openssh-server"
            echo "  sudo systemctl enable --now ssh"
        else
            local w_prl w_pa c_prl c_pa
            w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
            w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no
            c_prl=$(sshd_get PermitRootLogin); [ -z "$c_prl" ] && c_prl="(not set)"
            c_pa=$(sshd_get PasswordAuthentication); [ -z "$c_pa" ] && c_pa="(not set)"

            echo "Rule: Unset SSH options are NON-COMPLIANT (FAIL)."
            echo
            echo "[+] PermitRootLogin"
            echo "  Current: $c_prl"
            echo "  Policy : $w_prl"
            [ "$(echo "$c_prl"|tr A-Z a-z)" = "$(echo "$w_prl"|tr A-Z a-z)" ] && echo "    => PASS" || echo
            "    => FAIL"
            echo
            echo "[+] PasswordAuthentication"
            echo "  Current: $c_pa"
            echo "  Policy : $w_pa"
            [ "$(echo "$c_pa"|tr A-Z a-z)" = "$(echo "$w_pa"|tr A-Z a-z)" ] && echo "    => PASS" || echo
            "    => FAIL"
        fi
    } >"$T"
    log "Hardening: policy check"
    show "Policy Check" "$T"
}

```



```

}

apply_hardening() {
    ensure_policy
    is_root || { msg "Hardening" "Run with sudo to apply hardening."; return; }

    if ! ssh_installed; then
        msg "Hardening" "OpenSSH Server NOT installed.\nHardening is N/A.\n\nInstall:\n  sudo apt
install openssh-server\n  sudo systemctl enable --now ssh"
        return
    fi

    confirm "Hardening" "Will update SSH config based on policy.\nBackup will be
created.\nProceed?" || return

    local w_prl w_pa bak
    w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
    w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no

    bak=$(backup_sshd) || { msg "Hardening" "Backup failed. Aborting."; return; }

    sshd_set PermitRootLogin "$w_prl" || { msg "Hardening" "Failed to set PermitRootLogin";
return; }
    sshd_set PasswordAuthentication "$w_pa" || { msg "Hardening" "Failed to set
PasswordAuthentication"; return; }

    if sshd_validate; then
        if ssh_restart; then
            log "Hardening applied. Backup=$bak"
            msg "Hardening" "Hardening applied successfully.\nBackup:\n$bak"
        else
            cp "$bak" "$SSH_CFG" 2>/dev/null || true
            ssh_restart 2>/dev/null || true
            log "Hardening restart failed; rolled back. Backup=$bak"
            msg "Hardening" "SSH restart failed.\nRolled back:\n$bak"
        fi
    else
        cp "$bak" "$SSH_CFG" 2>/dev/null || true
        log "Hardening invalid config; rolled back. Backup=$bak"
        msg "Hardening" "Validation failed (sshd -t).\nRolled back:\n$bak"
    fi
}

rollback() {

```

```

ensure
is_root || { msg "Rollback" "Run with sudo to rollback."; return; }

if ! ssh_installed; then
    msg "Rollback" "OpenSSH Server NOT installed.\nRollback is N/A."
    return
fi

local b
b=$(ls -1 "$BACKUP_DIR"/sshd_config.*.bak 2>/dev/null | sort -r | head -n 15 || true)
[ -z "$b" ] && { msg "Rollback" "No backups found in:\n$BACKUP_DIR"; return; }

local items=() i=1 f
while IFS= read -r f; do items+=("$i" "$(basename "$f")"); i=$((i+1)); done <<<"$b"

local c sel
c=$(whiptail --title "Rollback" --menu "Select backup to restore:" 22 86 15 "${items[@]}" 3>&1
1>&2 2>&3) || return
sel=$(echo "$b" | sed -n "${c}p"); [ -z "$sel" ] && return

confirm "Rollback" "Restore this backup?\n\n$(basename "$sel")" || return
cp "$sel" "$SSH_CFG" 2>/dev/null || { msg "Rollback" "Restore failed."; return; }
sshd_validate && ssh_restart 2>/dev/null || true
log "Rollback restored $sel"
msg "Rollback" "Restored:\n$sel"
}

hardening_menu() {
    while true; do
        local c
        c=$(whiptail --title "SSH Hardening" --menu "$(status_line)\n\nSelect:" 20 86 10 \
            "1" "Policy PASS/FAIL (INI)" \
            "2" "Apply SSH Hardening (safe)" \
            "3" "Rollback SSH Config" \
            "4" "Back" 3>&1 1>&2 2>&3) || return
        case "$c" in
            1) policy_check ;;
            2) apply_hardening ;;
            3) rollback ;;
            4) return ;;
        esac
    done
}

```

```

# ----- Reports -----
report_generate() {
    ensure_policy; ensure
    local ts rep score=100
    ts=$(date +%F_%H-%M-%S)
    rep="$BACKUP_DIR/security_report_$(date +%F_%H-%M-%S).txt"

    local maxf maxd wwf wwd
    maxf=$(ini_get SCAN MaxWorldWritableFiles "$POLICY_FILE"); [ -z "$maxf" ] && maxf=50
    maxd=$(ini_get SCAN MaxWorldWritableDirs "$POLICY_FILE"); [ -z "$maxd" ] && maxd=50

    wwf=$(eval find / $FIND_PRUNE -type f -perm -0002 2>/dev/null | wc -l | tr -d ' ')
    wwd=$(eval find / $FIND_PRUNE -type d -perm -0002 2>/dev/null | wc -l | tr -d ' ')
    [ "$wwf" -gt "$maxf" ] && score=$((score-15))
    [ "$wwd" -gt "$maxd" ] && score=$((score-15))

    local ssh_note="" w_prl="" w_pa="" c_prl="" c_pa=""
    if ssh_installed; then
        w_prl=$(ini_get SSH PermitRootLogin "$POLICY_FILE"); [ -z "$w_prl" ] && w_prl=no
        w_pa=$(ini_get SSH PasswordAuthentication "$POLICY_FILE"); [ -z "$w_pa" ] && w_pa=no
        c_prl=$(sshd_get PermitRootLogin); [ -z "$c_prl" ] && c_prl="(not set)"
        c_pa=$(sshd_get PasswordAuthentication); [ -z "$c_pa" ] && c_pa="(not set)"
        [ "$(echo "$c_prl"|tr A-Z a-z)" != "$(echo "$w_prl"|tr A-Z a-z)" ] && score=$((score-20))
        [ "$(echo "$c_pa"|tr A-Z a-z)" != "$(echo "$w_pa"|tr A-Z a-z)" ] && score=$((score-20))
    else
        ssh_note="SSH NOT installed -> SSH checks are N/A (no scoring penalty)"
    fi
    [ "$score" -lt 0 ] && score=0

    {
        echo "Project: SentinelShield v1.1"
        echo "Purpose: Linux Security Audit & Hardening Framework"
        echo "Approach: Detect → Validate → Harden → Verify → Report"
        echo "Generated: $(date)"
        echo "Host: $(hostname)"
        echo "Policy: $POLICY_FILE"
        echo "Log: $LOGFILE"
        echo "-----"
        echo

        echo "[1] Permissions (REAL FS only)"
        echo "World-writable files: $wwf (max $maxf)"
        echo "World-writable dirs : $wwd (max $maxd)"
        echo
    }
}

```

```

echo "[2] SSH Security"
if ssh_installed; then
    echo "SSH installed: YES | Service: $(ssh_status)"
    echo "Rule: Unset SSH options are NON-COMPLIANT (FAIL)"
    echo "PermitRootLogin      : Current=$c_prl | Policy=$w_prl"
    echo "PasswordAuthentication : Current=$c_pa | Policy=$w_pa"
else
    echo "SSH installed: NO"
    echo "$ssh_note"
    echo "Install:"
    echo " sudo apt install openssh-server"
    echo " sudo systemctl enable --now ssh"
fi
echo

echo "[3] Ports (top 60)"
if have ss; then ss -tulpen 2>/dev/null | head -n 60
elif have netstat; then netstat -tulpen 2>/dev/null | head -n 60
else echo "No ss/netstat."; fi
echo

echo "[4] SUID/SGID (top 40)"
eval find / $FIND_PRUNE -type f \( -perm -4000 -o -perm -2000 \) 2>/dev/null | head -n 40
echo

echo "[5] Services (top 25)"
systemctl list-units --type=service --state=running 2>/dev/null | head -n 25 || true
echo
echo "=== FINAL SCORE: $score / 100 ==="
} >"$rep"

log "Report $rep score=$score"
msg "Report Generated" "Saved:\n$rep\n\nLog:\n$LOGFILE"
}

view_last_report() {
    local last
    last=$(ls -1 "$BACKUP_DIR"/security_report_*.txt 2>/dev/null | sort -r | head -n 1 || true)
    [ -z "$last" ] && { msg "Reports" "No report found in:\n$BACKUP_DIR"; return; }
    show "Last Report: $(basename "$last")" "$last"
}

view_log() {

```

```
[ -f "$LOGFILE" ] || { msg "Logs" "Log file not found yet."; return; }
show "Sentinel Log" "$LOGFILE"
}
```

```
reports_menu() {
while true; do
local c
c=$(whiptail --title "Reports & Logs" --menu "${status_line}\n\nSelect:" 20 86 10 \
  "1" "Generate Report + Score" \
  "2" "View Last Report" \
  "3" "View Log File" \
  "4" "Back" 3>&1 1>&2 2>&3) || return
case "$c" in
  1) report_generate ;;
  2) view_last_report ;;
  3) view_log ;;
  4) return ;;
esac
done
}
```

----- CPU Scheduling + renice -----

```
cpu_overview() {
bigmsg "CPU Module (OS Lab)" \
"Includes OS scheduling concepts:
- FCFS
- SJF (Non-preemptive)
- Round Robin (Quantum)
```

```
Also includes renice demo (nice value changes)."
}
```

```
cpu_renice_demo() {
local T1="$TMPDIR/sentinel_cpu_before.txt" T2="$TMPDIR/sentinel_cpu_after.txt"
ps -eo pid,pcpu,ni,comm --sort=-pcpu | head -n 15 >"$T1"
show "renice Demo - Before" "$T1"

local PID
PID=$(whiptail --inputbox "Enter PID to increase nice (+5):" 10 70 3>&1 1>&2 2>&3) || return
[ -z "$PID" ] && return

{
echo "=== renice Demo ==="
echo "Command: renice +5 $PID"
```

```

    echo
    renice +5 "$PID"
    echo
    ps -p "$PID" -o pid,pcpu,ni,comm
} >"$T2" 2>&1

log "renice demo PID=$PID"
show "renice Demo - After" "$T2"
}

PIDS=(); ARR=(); BUR=()
sched_get() {
    local N
    N=$(whiptail --inputbox "Processes count (1..12):" 10 60 "4" 3>&1 1>&2 2>&3) || return 1
    [[ "$N" =~ ^[0-9]+$ ]] && [ "$N" -ge 1 ] && [ "$N" -le 12 ] || { msg "Input" "Invalid N"; return 1; }
    PIDS=(); ARR=(); BUR=()
    local i a b
    for ((i=1;i<=N;i++)); do
        a=$(whiptail --inputbox "Arrival time for P$i (>=0):" 10 60 "0" 3>&1 1>&2 2>&3) || return 1
        b=$(whiptail --inputbox "Burst time for P$i (>=1):" 10 60 "5" 3>&1 1>&2 2>&3) || return 1
        [[ "$a" =~ ^[0-9]+$ ]] && [[ "$b" =~ ^[0-9]+$ ]] && [ "$b" -ge 1 ] || { msg "Input" "Bad input";
return 1; }
        PIDS+=("P$i"); ARR+=("$a"); BUR+=("$b")
    done
}

sort_arrival() { local n=${#PIDS[@]} i j t; IDX=(); for ((i=0;i<n;i++)); do IDX+=("$i"); done
    for ((i=0;i<n;i++)); do for ((j=0;j<n-1;j++)); do
        [ "${ARR[${IDX[j]}]}" -gt "${ARR[${IDX[j+1]}]}" ] && { t=${IDX[j]}; IDX[j]=${IDX[j+1]};
IDX[j+1]=t; }
    done; done
}

sched_out() {
    local title="$1" T="$TMPDIR/sched_result.txt"
    : >"$T"
    echo "=== CPU Scheduling: $title ===" >>"$T"
    echo >>"$T"
    echo "Gantt:" >>"$T"
    echo "$GANTT" >>"$T"
    echo >>"$T"
    printf "%-6s %-5s %-5s %-5s %-5s\n" "PID" "AT" "BT" "WT" "TAT" >>"$T"
    echo "-----" >>"$T"
    local n=${#PIDS[@]} sumw=0 sumt=0 i
    for ((i=0;i<n;i++)); do

```

```

    printf "%-6s %-5s %-5s %-5s %-5s\n" "${PIDS[i]}" "${ARR[i]}" "${BUR[i]}" "${WT[i]}" "${TAT[i]}"
>>"$T"
    sumw=$((sumw+WT[i])); sumt=$((sumt+TAT[i]))
done
echo >>"$T"
echo "Avg WT = $(awk "BEGIN{printf \"%.2f\", $sumw/$n}")" >>"$T"
echo "Avg TAT = $(awk "BEGIN{printf \"%.2f\", $sumt/$n}")" >>"$T"
log "Scheduling $title"
show "Scheduling Result" "$T"
}
fcfs() {
    local n=${#PIDS[@]} i t=0; WT=(); TAT=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; done
    sort_arrival; GANTT="(time)"
    for idx in "${IDX[@]}; do
        [ "$t" -lt "${ARR[idx]}" ] && t=${ARR[idx]}
        WT[idx]=$((t-ARR[idx])); t=$((t+BUR[idx])); TAT[idx]=$((t-ARR[idx]))
        GANTT="$GANTT | ${PIDS[idx]} ($t)"
    done
    sched_out "FCFS"
}
sjf_np() {
    local n=${#PIDS[@]} i; WT=(); TAT=(); DONE=()
    for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; DONE[i]=0; done
    sort_arrival; local t=${ARR[${IDX[0]}]} done=0; GANTT="(time)"
    while [ "$done" -lt "$n" ]; do
        local pick=-1 best=999999
        for ((i=0;i<n;i++)); do
            [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && [ "${BUR[i]}" -lt "$best" ] && { pick=$i;
best=${BUR[i]}; }
        done
        if [ "$pick" -eq -1 ]; then
            local next=999999
            for ((i=0;i<n;i++)); do [ "${DONE[i]}" -eq 0 ] && [ "${ARR[i]}" -lt "$next" ] && next=${ARR[i]};
done
            t=$next; continue
        fi
        WT[pick]=$((t-ARR[pick])); t=$((t+BUR[pick])); TAT[pick]=$((t-ARR[pick]))
        DONE[pick]=1; done=$((done+1)); GANTT="$GANTT | ${PIDS[pick]} ($t)"
    done
    sched_out "SJF (Non-preemptive)"
}
rr() {

```

```

local q; q=$(whiptail --inputbox "Round Robin quantum (>=1):" 10 60 "2" 3>&1 1>&2 2>&3) ||
return
[[ "$q" =~ ^[0-9]+$ ]] && [ "$q" -ge 1 ] || { msg "Input" "Invalid quantum"; return; }
local n=${#PIDS[@]}; i=0; WT=(); TAT=(); REM=(); FIN=(); ENQ=()
for ((i=0;i<n;i++)); do WT[i]=0; TAT[i]=0; REM[i]={BUR[i]}; FIN[i]=0; ENQ[i]=0; done
sort_arrival; local t=${ARR[${IDX[0]}]} queue=() head=0 done=0; GANTT="(time)"
for ((i=0;i<n;i++)); do [ "${ARR[i]}" -le "$t" ] && [ "${ENQ[i]}" -eq 0 ] && { queue+=("$i");
ENQ[i]=1; }; done
while [ "$done" -lt "$n" ]; do
    if [ "$head" -ge "${#queue[@]}" ]; then
        local next=999999
        for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -lt "$next" ]
&& next=${ARR[i]}; done
        t=$next
        for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && {
queue+=("$i"); ENQ[i]=1; }; done
        head=0; continue
    fi
    local p=${queue[head]}; head=$((head+1))
    local run=$q; [ "${REM[p]}" -lt "$run" ] && run=${REM[p]}
    t=$((t+run)); REM[p]=$((REM[p]-run)); GANTT="$GANTT | ${PIDS[p]} (+$run=>$t)"
    for ((i=0;i<n;i++)); do [ "${FIN[i]}" -eq 0 ] && [ "${ENQ[i]}" -eq 0 ] && [ "${ARR[i]}" -le "$t" ] && {
queue+=("$i"); ENQ[i]=1; }; done
    if [ "${REM[p]}" -eq 0 ]; then FIN[p]=1; done=$((done+1)); TAT[p]=$((t-ARR[p])); else
queue+=("$p"); fi
    [ "$head" -gt 25 ] && { queue=("${queue[@]:head}"); head=0; }
done
for ((i=0;i<n;i++)); do WT[i]=$((TAT[i]-BUR[i])); done
sched_out "Round Robin (q=$q)"
}

cpu_alg_menu() {
    sched_get || return
    while true; do
        local c
        c=$(whiptail --title "CPU Scheduling" --menu "Select algorithm:" 18 70 10 \
            "1" "FCFS" \
            "2" "SJF (Non-preemptive)" \
            "3" "Round Robin" \
            "4" "Back" 3>&1 1>&2 2>&3) || return
        case "$c" in
            1) fcfs ;;
            2) sjf_np ;;
            3) rr ;;
            4) return ;;
        esac
    done
}

```



```

        esac
    done
}
cpu_menu() {
    while true; do
        local c
        c=$(whiptail --title "CPU Module (OS Lab)" --menu "$(status_line)\n\nSelect:" 20 86 10 \
            "1" "Overview" \
            "2" "Algorithms (FCFS/SJF/RR)" \
            "3" "renice Demo" \
            "4" "Back" 3>&1 1>&2 2>&3) || return
        case "$c" in
            1) cpu_overview ;;
            2) cpu_alg_menu ;;
            3) cpu_renice_demo ;;
            4) return ;;
        esac
    done
}

# ----- Main -----
main_menu() {
    need whiptail
    ensure_policy
    ensure
    intro
    env_check

    while true; do
        local c
        c=$(whiptail --title "SentinelShield - Main Menu" --menu "$(status_line)\n\nChoose a module:"
20 86 10 \
            "1" "Security Audit" \
            "2" "SSH Hardening (Policy + Apply + Rollback)" \
            "3" "Reports & Logs" \
            "4" "CPU Scheduling (OS Lab)" \
            "5" "Exit" 3>&1 1>&2 2>&3)

        [ $? -ne 0 ] && clear && exit 0
        case "$c" in
            1) audit_menu ;;
            2) hardening_menu ;;
            3) reports_menu ;;
            4) cpu_menu ;;

```

```
        5) clear; exit 0 ;;
    esac
done
}

main_menu
```